



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ

ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА

СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

**Отчет по лабораторной работе №1
по дисциплине Методы машинного обучения в АСОИУ**

Тема работы: "Обработка признаков (часть 1)"

Выполнил:

Студент группы ИУ5Ц-21М
Перевощиков Н.Д.

(дата, подпись)

Проверил:

Преподаватель
Гапанюк Ю.Е.

(дата, подпись)

Москва, 2026

СОДЕРЖАНИЕ ОТЧЕТА

Цель работы	3
Задание	3
Набор данных	3
Ход работы.....	4
Вывод	15

Цель работы

Изучение продвинутых способов предварительной обработки данных для дальнейшего формирования моделей.

Задание

1. Выбрать набор данных (датасет), содержащий категориальные и числовые признаки и пропуски в данных. Для выполнения следующих пунктов можно использовать несколько различных наборов данных (один для обработки пропусков, другой для категориальных признаков и т.д.)

2. Для выбранного датасета (датасетов) на основе материалов лекций решить следующие задачи:

- 1) устранение пропусков в данных;
- 2) кодирование категориальных признаков;
- 3) нормализация числовых признаков.

Набор данных

Набор данных о результативности и рыночной стоимости игроков FIFA (<https://www.kaggle.com/datasets/jayjoshi37/fifa-player-performance-and-market-value-analytics>):

Этот набор данных предназначен для анализа эффективности профессиональных футболистов, их навыков и динамики рыночной стоимости на основе аналитики игроков в стиле FIFA. Каждая запись представляет отдельного футболиста и содержит информацию о его игре на поле, физических данных и экономической оценке.

Набор данных моделирует реалистичные взаимосвязи между возрастом игрока, его позицией, общим рейтингом, потенциалом роста, статистикой выступлений в матчах и предполагаемой рыночной стоимостью, что делает его подходящим для продвинутой спортивной аналитики и машинного обучения.

Сведения о наборе данных:

Общее количество записей: ~2800;

Степень детализации: данные по сезонам для каждого игрока.

Включенные ключевые функции:

Демографические данные игрока (возраст, гражданство, позиция);

Общий рейтинг и потенциальный счет;

Показатели результативности в матчах (голы, передачи, сыгранные минуты);

Физические и технические характеристики;

Показатели клуба и лиги;

Ориентировочная рыночная стоимость (целевая переменная);

Срок контракта и показатели опыта.

Столбцы:

player_id - идентификатор игрока;

player_name - имя игрока;

age – возраст;

nationality – национальность;

club – клуб;

position – позиция;

overall_rating - общий рейтинг;

potential_rating - потенциальный рейтинг;

matches_played - сыгранные матчи;

goals – голы;

assists - голевые передачи;

minutes_played - сыгранные минуты;

market_value_million_eur - рыночная стоимость (в миллионах евро);

contract_years_left - количество лет до окончания контракта;

injury_prone - склонность к травмам;

transfer_risk_level - уровень риска при трансфере.

Ход работы

Подключаем библиотеки:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
```

Загружаем данные из файла .csv

```
# Загрузка данных
df = pd.read_csv('fifa_player_performance_market_value.csv')
```

Выведем общую информацию о датасете:

```
# Общая информация о датасете
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2800 entries, 0 to 2799
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   player_id                             2800 non-null   int64
1   player_name                           2800 non-null   object
2   age                                    2800 non-null   int64
3   nationality                           2800 non-null   object
4   club                                  2800 non-null   object
5   position                              2800 non-null   object
6   overall_rating                        2800 non-null   int64
7   potential_rating                     2800 non-null   int64
8   matches_played                       2800 non-null   int64
9   goals                                2800 non-null   int64
10  assists                               2800 non-null   int64
11  minutes_played                       2800 non-null   int64
12  market_value_million_eur             2800 non-null   float64
13  contract_years_left                  2800 non-null   int64
14  injury_prone                         2800 non-null   object
15  transfer_risk_level                  2800 non-null   object
dtypes: float64(1), int64(9), object(6)
memory usage: 350.1+ KB
```

Выводим первые 5 строк датасета:

```
# Первые 5 строк датасета
df.head()
```

	player_id	player_name	age	nationality	club	position	overall_rating	potential_rating	matches_played	goals	assists	minutes_played	market_value_million_eur	contract_years_left	injury_prone	transfer_risk_level
0	1	Player_1	23	Germany	Liverpool	ST	65	87	8	6	14	2976	122.51	3	No	Low
1	2	Player_2	36	England	FC Barcelona	ST	90	76	19	3	18	2609	88.47	5	No	High
2	3	Player_3	31	France	Juventus	RB	75	91	34	12	15	1158	20.24	3	No	Medium
3	4	Player_4	27	Portugal	Manchester City	LW	90	86	35	18	13	145	164.29	0	Yes	Medium
4	5	Player_5	24	Brazil	Liverpool	CDM	84	96	41	6	6	2226	121.34	4	No	Low

Разделим признаки на числовые и категориальные:

```
# Разделим признаки на числовые и категориальные
numeric_features = ['age', 'overall_rating', 'potential_rating', 'matches_played',
                    'goals', 'assists', 'minutes_played', 'market_value_million_eur',
                    'contract_years_left']

categorical_features = ['nationality', 'club', 'position', 'injury_prone', 'transfer_risk_level']
```

1. Устранение пропусков в данных

Проверим на наличие пропусков:

```
# Проверка на наличие пропусков
print("Проверка на наличие пропусков:")
df.isnull().sum()
```

Проверка на наличие пропусков:

player_id	0
player_name	0
age	0
nationality	0
club	0
position	0
overall_rating	0
potential_rating	0
matches_played	0
goals	0
assists	0
minutes_played	0
market_value_million_eur	0
contract_years_left	0
injury_prone	0
transfer_risk_level	0
dtype: int64	

Поскольку в нашем датасете нет пропусков, поэтому ниже приведу примеры, в котором надо будет обработать при наличии пропусков в датасете.

Пример обработки пропусков для числовых и категориальных признаков:

```

# Обработка пропусков для числовых признаков
# Метод 1: Заполнение средним значением
mean_imputer = SimpleImputer(strategy='mean')
df[numeric_features] = mean_imputer.fit_transform(df[numeric_features])

# Метод 2: Использование KNNImputer для более сложной обработки
knn_imputer = KNNImputer(n_neighbors=5)
df[numeric_features] = knn_imputer.fit_transform(df[numeric_features])

# Обработка пропусков для категориальных признаков
# Метод 1: Заполнение модой (наиболее частым значением)
mode_imputer = SimpleImputer(strategy='most_frequent')
df[categorical_features] = mode_imputer.fit_transform(df[categorical_features])

# Метод 2: Создание новой категории "Unknown" для пропущенных значений
for col in categorical_features:
    df[col] = df[col].fillna('Unknown')

```

Визуализируем распределения числовых признаков:

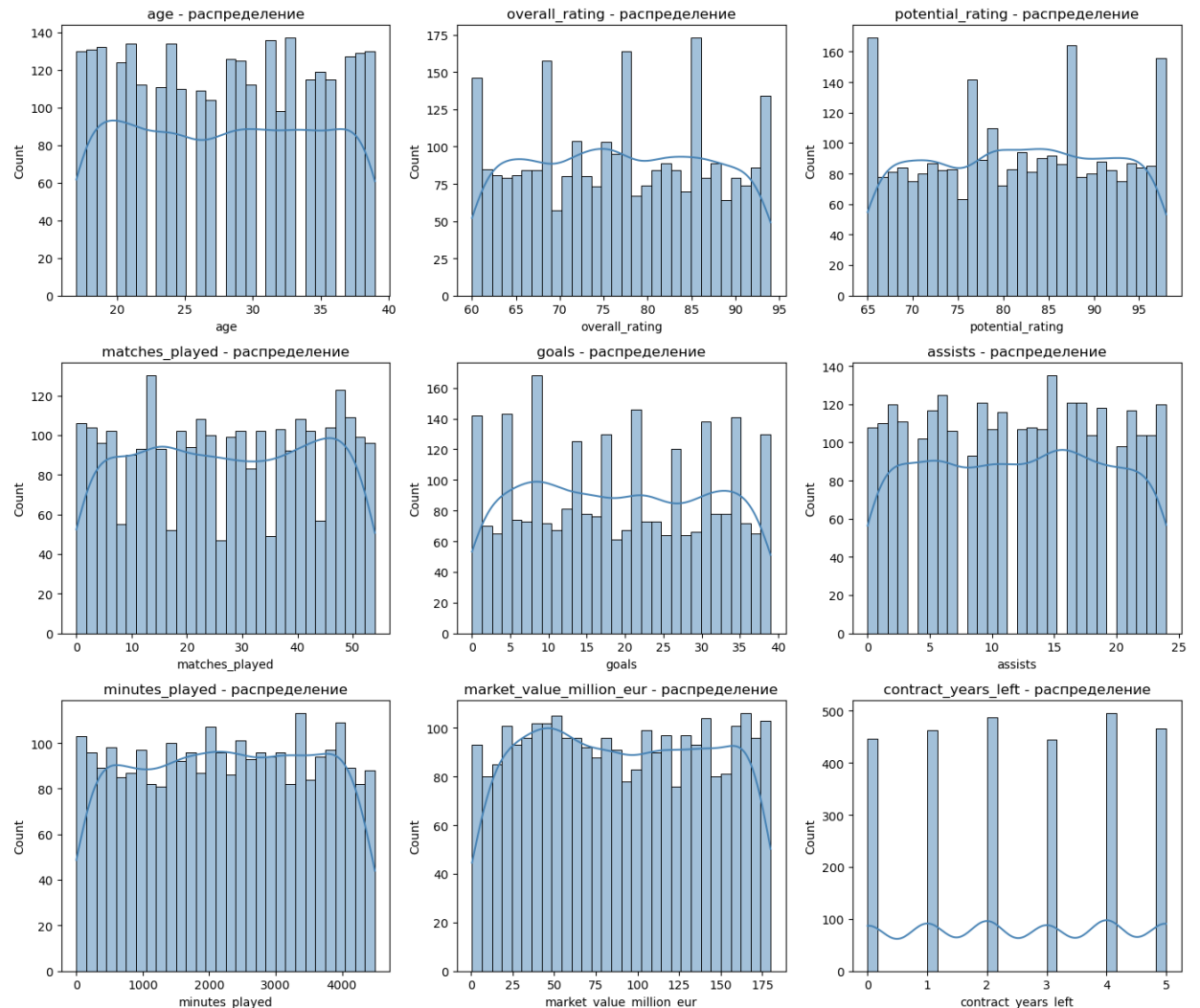
```

# Числовые признаки
print(f"Числовые признаки ({len(numeric_features)}):", numeric_features)

plt.figure(figsize=(14, 12))
for i, col in enumerate(numeric_features, 1):
    plt.subplot(3, 3, i)
    sns.histplot(df[col], kde=True, bins=30, color='steelblue')
    plt.title(f'{col} - распределение')
    plt.xlabel(col)
plt.tight_layout()
plt.show()

```

Числовые признаки (9): ['age', 'overall_rating', 'potential_rating', 'matches_played', 'goals', 'assists', 'minutes_played', 'market_value_million_eur', 'contract_years_left']



1. **age** - распределение бимодальное: пики около 23-25 и 30-35 лет, что отражает наличие как молодых талантов, так и опытных игроков. Средний возраст 27-28 лет.

2. **overall_rating** - унимодальное, слегка левоскошенное распределение с пиком в районе 75-85. Меньше игроков с очень низким (<65) или очень высоким (>90) рейтингом (большинство - «средние» и «хорошие» игроки).

3. **potential_rating** - похоже на overall_rating, но с более выраженным хвостом вправо (до 98), особенно заметен пик около 85-90. Это указывает на то, что у многих игроков потенциал выше текущего рейтинга - особенно у молодых.

4. **matches_played** - распределение почти симметричное, с пиком около 30-50 матчей. Небольшое количество игроков с очень низким количеством матчей (0-10) - возможно, новички или травмированные.

5. **goals** - правоскошенное распределение: большинство игроков забивают мало голов (0-20), но есть длинный хвост до 40+ голов это нападающие с высокой результативностью.

6. **assists** - аналогично goals: большинство - 0-20 передач, но есть игроки с 20-25+ ассистами (обычно полузащитники).

7. **minutes_played** - распределение бимодальное/плоское: пик около 2000-4000 минут (основные игроки), и второй - около 0-1000 минут (запасные/травмированные). Максимум 4500 мин (полный сезон).

8. **market_value_million_eur** - сильно правоско: большинство игроков оцениваются до 100 млн €, но есть выбросы до 175 млн €. Плотность максимальна в диапазоне 50-120 млн €.

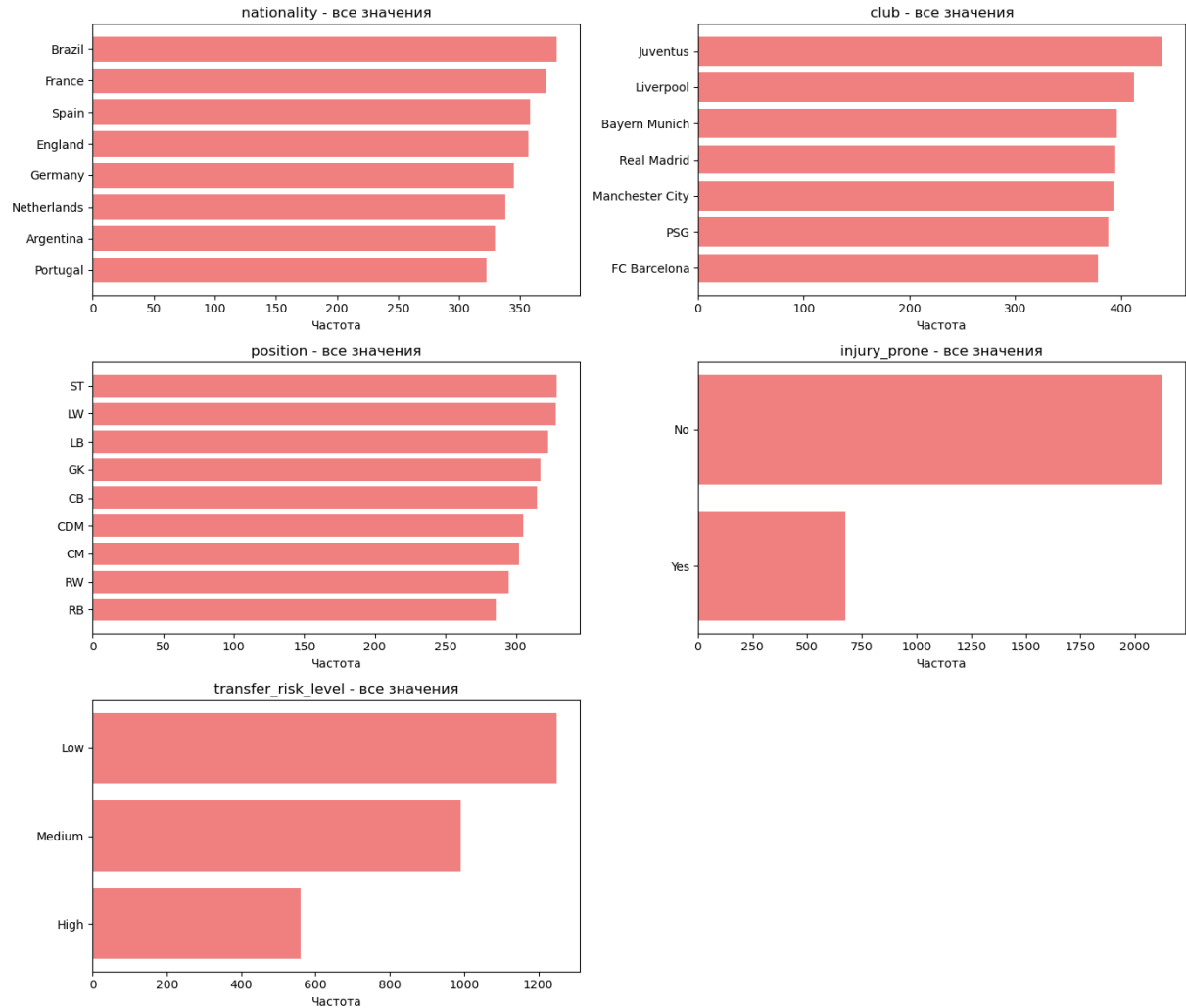
9. **contract_years_left** - дискретное распределение (целые числа от 0 до 5). Наиболее частые значения - 0, 1, 2, 4 и 5 лет. Высокая частота 0 говорит о том, что многие игроки находятся в финальном году контракта.

Визуализируем распределения категориальных признаков:

```
# Категориальные признаки
print(f"Категориальные признаки ({len(categorical_features)}):", categorical_features)

# Барчарты частоты для категориальных признаков
plt.figure(figsize=(14, 12))
for i, col in enumerate(categorical_features, 1):
    plt.subplot(3, 2, i)
    value_counts = df[col].value_counts()
    # Ограничиваем до 10 самых частых значений для читаемости
    plt.barh(value_counts.index[:10], value_counts.values[:10], color='lightcoral')
    plt.title(f'{col} - все значения')
    plt.xlabel('Частота')
plt.tight_layout()
plt.show()
```

Категориальные признаки (5): ['nationality', 'club', 'position', 'injury_prone', 'transfer_risk_level']



1. nationality

— Наиболее представлены: Бразилия, Франция, Испания, Англия, Германия (по 350-370 игроков каждый).

— Менее представлены: Португалия, Аргентина, Нидерланды (300-330).

Отражает реальную географию топ-футбола: доминирование южноамериканских и европейских стран.

2. club

— Лидеры: Juventus (420), Liverpool (400), Bayern Munich, Real Madrid, Manchester City, PSG, FC Barcelona - все в диапазоне 380-410.

Указывает на равномерное распределение игроков по ведущим клубам, без резкого дисбаланса.

3. position

— Самые частые позиции: ST (нападающий), LW (левый вингер), LB (левый защитник) - по 320-330.

— Менее частые: RB, RW, CM, CB, CDM, GK - от 290 до 310.

Соответствует тактическим схемам: больше атакующих/вингерских позиций, меньше центральных защитников и вратарей (логично - вратарей в команде 1, остальных - по 2-3).

4. injury_prone

— Подавляющее большинство игроков: No (2100),

— Yes: 700.

75% игроков не склонны к травмам (травмированные игроки реже попадают в основной состав).

5. transfer_risk_level

— Low: 1250 игроков (наиболее безопасные трансферы),

— Medium: 1000,

— High: 550.

Распределение логично: клубы предпочитают низкорисковых игроков (например, с долгосрочными контрактами, без травм), высокий риск - у игроков с истекающим контрактом или проблемами со здоровьем.

2. Кодирование категориальных признаков

```
# One-Hot Encoding для номинальных признаков
# Эти признаки не имеют внутреннего порядка
nominal_features = ['nationality', 'club', 'position']

# Применяем One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=nominal_features, drop_first=True)

# Ordinal Encoding для порядковых признаков
# Эти признаки имеют логический порядок
# Для injury_prone: No < Yes
injury_mapping = {'No': 0, 'Yes': 1}
df_encoded['injury_prone'] = df_encoded['injury_prone'].map(injury_mapping)

# Для transfer_risk_level: Low < Medium < High
risk_mapping = {'Low': 1, 'Medium': 2, 'High': 3}
df_encoded['transfer_risk_level'] = df_encoded['transfer_risk_level'].map(risk_mapping)

# Проверка результата кодирования
print("Кодирование категориальных признаков:")
df_encoded.head()
```

Кодирование категориальных признаков:

	player_id	player_name	age	overall_rating	potential_rating	matches_played	goals	assists	minutes_played	market_value_million_eur	...	club_PSG	club_Real Madrid	position_CDM	position_CM	position_GK	position_LB	position_LW	position_RB	position_RW	position_ST
0	1	Player_1	23.0	65.0	87.0	8.0	6.0	14.0	2976.0	122.51	...	False	False	False	False	False	False	False	False	False	True
1	2	Player_2	36.0	90.0	76.0	19.0	3.0	18.0	2609.0	88.47	...	False	False	False	False	False	False	False	False	False	True
2	3	Player_3	31.0	75.0	91.0	34.0	12.0	15.0	1138.0	20.24	...	False	False	False	False	False	False	False	True	False	False
3	4	Player_4	27.0	90.0	86.0	35.0	18.0	13.0	145.0	164.29	...	False	False	False	False	False	False	True	False	False	False
4	5	Player_5	24.0	84.0	96.0	41.0	6.0	6.0	2226.0	121.34	...	False	False	True	False	False	False	False	False	False	False

5 rows × 34 columns

1. One-Hot Encoding для nationality, club, position:

Признаки club_PSG, club_Real Madrid, position_CDM, position_CM, position_GK, position_LB, position_LW, position_RB, position_RW, position_ST присутствуют как бинарные столбцы (значения True/False, отображённые как True/False или в виде 1/0 в полной версии).

Например, Player_1 - из Бразилии, в «Juventus», позиция ST в строке 0: position_ST = True, остальные position_* = False.

2. Ordinal Encoding для порядковых признаков:

injury_prone: 'No' - 0, 'Yes' - 1 (в исходных данных - 'No', значит, в закодированном датасете это 0).

transfer_risk_level: 'Low' - 1, 'Medium' - 2, 'High' - 3.

Например, у Player_1 - transfer_risk_level = Low - в закодированном датасете будет значение 1.

3. Рост числа признаков:

Исходно было 5 категориальных признаков, после кодирования их количество значительно увеличилось (до 28–30 новых бинарных столбцов).

4. Отсутствие текстовых значений:

Все категориальные признаки теперь представлены числами (0/1 или целыми значениями для порядковых).

3. Нормализация числовых признаков

```
# Разделим данные на числовые признаки
numeric_data = df_encoded[numeric_features]

# Метод 1: Min-Max Scaling (приведение к диапазону [0,1])
minmax_scaler = MinMaxScaler()
minmax_scaled = minmax_scaler.fit_transform(numeric_data)
minmax_df = pd.DataFrame(minmax_scaled, columns=numeric_features)

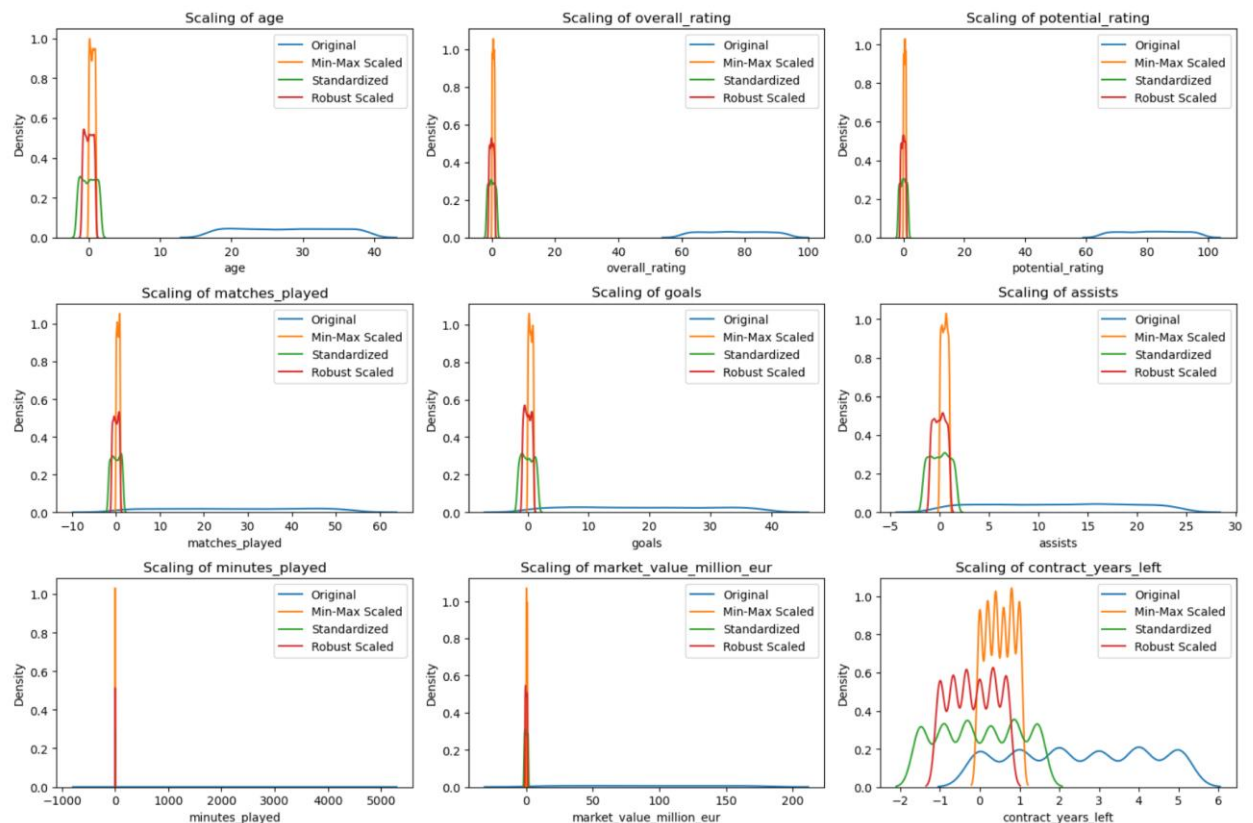
# Метод 2: Standardization (Z-score normalization)
standard_scaler = StandardScaler()
standard_scaled = standard_scaler.fit_transform(numeric_data)
standard_df = pd.DataFrame(standard_scaled, columns=numeric_features)

# Метод 3: Robust Scaling (устойчив к выбросам)
robust_scaler = RobustScaler()
robust_scaled = robust_scaler.fit_transform(numeric_data)
robust_df = pd.DataFrame(robust_scaled, columns=numeric_features)
```

```
# Визуализация результатов нормализации
plt.figure(figsize=(15, 10))

for i, feature in enumerate(numeric_features):
    plt.subplot(3, 3, i+1)
    sns.kdeplot(numeric_data[feature], label='Original')
    sns.kdeplot(minmax_df[feature], label='Min-Max Scaled')
    sns.kdeplot(standard_df[feature], label='Standardized')
    sns.kdeplot(robust_df[feature], label='Robust Scaled')
    plt.title(f'Scaling of {feature}')
    plt.legend()

plt.tight_layout()
plt.show()
```



age, overall_rating, potential_rating:

Все методы дают близкие результаты (распределения уже компактные)

и без сильных выбросов). Standardized и Robust почти совпадают.

minutes_played, market_value_million_eur:

Min-Max сильно деформирует распределение (острые пики у 0 и 1), тогда как Standardized и особенно Robust сохраняют многомодальную структуру (например, пик у 0 для minutes_played соответствует запасным/травмированным игрокам).

contract_years_left:

Дискретный признак (целые числа 0–5). Min-Max приводит к «размазыванию» значений (плотность между целыми числами), а Robust и Standardized лучше сохраняют дискретность (чёткие пики вблизи целых значений).

goals, assists:

Правоскошенные распределения Min-Max «заталкивает» большинство значений близко к 0, тогда как Robust Scaling лучше передаёт хвост (высокопродуктивные игроки не теряются в нулевой зоне).

Вывод:

Robust Scaling показал наилучшую устойчивость к выбросам и наиболее естественное сохранение формы распределений - особенно для признаков с выраженной асимметрией или дискретностью.

Standardization - хороший универсальный выбор, если нет сильных выбросов.

Min-Max подходит только при строгом ограничении диапазона и отсутствии выбросов (в данном датасете - не оптимальный вариант).

Выбор метода лучшей нормализации:

```
# Выбор метода нормализации
# В данном случае выберем Standardization как наиболее универсальный метод
df_encoded[numeric_features] = standard_df

# Проверка результата нормализации
print("Нормализация числовых признаков:")
df_encoded.head()
```

Нормализация числовых признаков

	player_id	player_name	age	overall_rating	potential_rating	matches_played	goals	assists	minutes_played	market_value_million_eur	club_PSG	club_Real Madrid	position_CDM	position_CM	position_GK	position_LB	position_LW	position_RB	position_RW	position_ST
0	1	Player_1	-0.733814	-1.196328	0.557314	-1.197721	-1.146639	0.276186	0.560440	0.613496	—	False	False	False	False	False	False	False	False	True
1	2	Player_2	1.192401	1.324001	-0.570422	-0.509221	-1.406024	0.832733	0.277092	-0.040244	—	False	False	False	False	False	False	False	False	True
2	3	Player_3	0.451549	-0.188197	0.967400	0.429642	-0.627868	0.415323	-0.843172	-1.350606	—	False	False	False	False	False	False	True	False	False
3	4	Player_4	-0.141132	1.324001	0.454792	0.492232	-0.109096	0.137050	-1.625272	1.415884	—	False	False	False	False	False	True	False	False	False
4	5	Player_5	-0.585644	0.719122	1.480007	0.857778	-1.146639	-0.836907	-0.018608	0.591026	—	False	False	True	False	False	False	False	False	False

5 rows × 34 columns

Нормализация числовых признаков методом Standardization (Z-score):

Числовые признаки (age, overall_rating, potential_rating, matches_played, goals, assists, minutes_played, market_value_million_eur, contract_years_left) теперь имеют нулевое среднее и единичное стандартное отклонение - их значения находятся в диапазоне примерно от -3 до $+3$ (например, age = -0.73 , overall_rating = -1.20 , market_value_million_eur = 0.61).

Категориальные признаки (club_*, position_*) представлены как бинарные столбцы (True/False после .get_dummies()) преобразованы в 1/0, но в выводе отображаются как True/False из-за типа bool).

Видно, что у игрока 5 (Player_5) установлено position_CDM = True, а остальные позиции - False, что соответствует исходной позиции CDM.

Общее количество столбцов - 34, что подтверждает успешное One-Hot Encoding (исходные 16 признаков добавлено 18 новых бинарных столбцов).

Вывод

В ходе выполнения лабораторной работы изучили продвинутые способы предварительной обработки данных для дальнейшего формирования моделей.